



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Andrews, Kurt Kent C.
Section:	AN41
Professor:	Ms. Crisola Tan
Date:	10/03/2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
(Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
 - Classes and objects

- If-Else statements
 - Loops (for loop)
 - Functions / methods
2. Understand the environment:
- Two rooms: A and B
 - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output

```
C:\Users\hazel\AppData\Local... × + ▾  
Is room A Dirty or Clean? dirty  
Is room B Dirty or Clean? clean  
  
How many steps should the agent run? 5  
  
Step 1: Agent is in room A  
Room A has been cleaned.  
Environment state: {'A': 'Clean', 'B': 'Clean'}  
  
Step 2: Agent is in room A  
Moving to room B  
Environment state: {'A': 'Clean', 'B': 'Clean'}  
  
Step 3: Agent is in room B  
Moving to room A  
Environment state: {'A': 'Clean', 'B': 'Clean'}  
  
Step 4: Agent is in room A  
Moving to room B  
Environment state: {'A': 'Clean', 'B': 'Clean'}  
  
Step 5: Agent is in room B  
Moving to room A  
Environment state: {'A': 'Clean', 'B': 'Clean'}  
Press any key to continue . . . |
```

Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
 - Rule 1: If the current room is dirty, clean it.
 - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
 - The agent's current location
 - The action taken (cleaned/moved)
 - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)

```
C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Dirty] [C-Dirty]
Moving to room B
[A-Clean] [B-Agent] [C-Dirty]

Step 2:
[A-Clean] [B-Agent] [C-Dirty]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Dirty]

Step 3:
[A-Clean] [B-Agent] [C-Dirty]
Moving to room C
[A-Clean] [B-Clean] [C-Agent]

Step 4:
[A-Clean] [B-Clean] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]

Step 5:
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
Press any key to continue . . . |
```

Your Code - Laboratory Tasks

Tasks 1 - 3 ([RuleBasedAgent.py](#) on Github)

```
Is room A Dirty or Clean? idk
Please enter Dirty or Clean.
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? DIRTY
```

```
How many steps should the agent run? 5
```

```
Initial Environment State: {'A': 'Dirty', 'B': 'Dirty'}
```

```
Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Dirty'}
```

```
Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Dirty'}
```

```
Step 3: Agent is in room B
Room B has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}
```

```
Step 4: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
```

```
Step 5: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}
```

```
def normalize_room_state(state):
    state = state.strip().lower()

    if state == "dirty":
        return "Dirty"
    if state == "clean":
        return "Clean"

    return None
```

```

def ask_room_state(room):
    while True:
        state = input(f"Is room {room} Dirty or Clean? ")
        normalized_state = normalize_room_state(state)

        if normalized_state is not None:
            return normalized_state

        print("Please enter Dirty or Clean.")

def is_room_dirty(environment, room):
    return environment[room] == "Dirty"

def clean_room(environment, room):
    environment[room] = "Clean"

class RuleBasedAgent:
    def __init__(self, start_location="A"):
        self.location = start_location

    def perceive_and_act(self, environment):
        if is_room_dirty(environment, self.location):
            clean_room(environment, self.location)
            return f"Room {self.location} has been cleaned."

        return self.move()

    def move(self):
        if self.location == "A":
            self.location = "B"
        else:
            self.location = "A"

        return f"Moving to room {self.location}"

def ask_steps():
    while True:
        try:
            steps = int(input("\nHow many steps should the agent run? "))

```

```

        if steps > 0:
            return steps

        print("Please enter a positive number.")
    except ValueError:
        print("Please enter a valid number.")

def main():
    environment = {
        "A": ask_room_state("A"),
        "B": ask_room_state("B"),
    }

    agent = RuleBasedAgent()
    steps = ask_steps()

    print(f"\nInitial Environment State: {environment}")
    for step in range(1, steps + 1):
        print(f"\nStep {step}: Agent is in room {agent.location}")
        action = agent.perceive_and_act(environment)
        print(action)
        print(f"Environment state: {environment}")

if __name__ == "__main__":
    main()

```

Optional Task ([RuleBasedAgentExtra.py](#) on Github)

```
Is room A Dirty or Clean? Dirty
Is room B Dirty or Clean? Clean
Is room C Dirty or Clean? Dirty
```

How many steps should the agent run? 5

Step 1:

```
[A-Agent] [B-Clean] [C-Dirty]
Room A has been cleaned.
[A-Agent] [B-Clean] [C-Dirty]
```

Step 2:

```
[A-Agent] [B-Clean] [C-Dirty]
Moving to room C
[A-Clean] [B-Clean] [C-Agent]
```

Step 3:

```
[A-Clean] [B-Clean] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]
```

Step 4:

```
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
```

Step 5:

```
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
```

```
import random
```

```
ROOMS = ["A", "B", "C"]
```

```
def normalize_room_state(state):
    state = state.strip().lower()
```

```

    if state == "dirty":
        return "Dirty"
    if state == "clean":
        return "Clean"

    return None

def ask_room_state(room):
    while True:
        state = input(f"Is room {room} Dirty or Clean? ")
        normalized_state = normalize_room_state(state)

        if normalized_state is not None:
            return normalized_state

        print("Please enter Dirty or Clean.")

def is_room_dirty(environment, room):
    return environment[room] == "Dirty"

def clean_room(environment, room):
    environment[room] = "Clean"

def show_grid(environment, agent_location):
    room_display = []

    for room in ROOMS:
        if room == agent_location:
            room_display.append(f"[{room}-Agent]")
        else:
            room_display.append(f"[{room}-{environment[room]}]")

    print(" ".join(room_display))

class RuleBasedAgent:
    def __init__(self, start_location="A"):
        self.location = start_location

    def perceive_and_act(self, environment):
        if is_room_dirty(environment, self.location):

```

```

        clean_room(environment, self.location)
        return f"Room {self.location} has been cleaned."

    dirty_rooms = [
        room for room in ROOMS
        if room != self.location and is_room_dirty(environment, room)
    ]

    if dirty_rooms:
        next_room = random.choice(dirty_rooms)
        return self.move(next_room)

    return "All rooms are clean. Agent is idle."

    def move(self, room):
        self.location = room
        return f"Moving to room {self.location}"

def ask_steps():
    while True:
        try:
            steps = int(input("\nHow many steps should the agent run? "))

            if steps > 0:
                return steps

            print("Please enter a positive number.")
        except ValueError:
            print("Please enter a valid number.")

def main():
    environment = {
        room: ask_room_state(room)
        for room in ROOMS
    }

    agent = RuleBasedAgent()
    steps = ask_steps()

    for step in range(1, steps + 1):
        print(f"\nStep {step}:")
        show_grid(environment, agent.location)
        action = agent.perceive_and_act(environment)

```

```
print(action)
show_grid(environment, agent.location)

if __name__ == "__main__":
    main()
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?

The agent follows simple condition-based rules. If the current room is dirty, it cleans that room. If the current room is clean, it moves to another room. These rules help the agent decide what action to take at each step.

2. How does the agent decide when to clean and when to move?

The agent first checks the state of the room where it is located. If that room is dirty, the agent cleans it. If the room is already clean, the agent moves to another room. This is called a perceive-and-act process.

3. What happens if both rooms are already clean at the start?

If both rooms are already clean at the start, the agent does not need to clean anything. In a basic two-room version, it may continue moving between the rooms. In an improved version, it can stop or stay idle. This shows that an idle rule can make the agent more efficient.

4. How would you modify the agent to handle **more than two rooms**?

To handle more than two rooms, the rooms can be stored in a dictionary or list. The agent's movement logic should be changed so it can choose from multiple rooms. It can move to the next room in order or randomly select a dirty room. This makes the agent more flexible for larger environments.

Answer the Questions 

